Adaptive Kalman Filter

Based on:

- *Kalman, R. E. (1960). "A New Approach to Linear Filtering and Prediction Problems.“*
- *Akhlaghi, S., et al. (2017). "Adaptive Adjustment of Noise Covariance in Kalman Filter for Dynamic State Estimation."*

Student: Ramez Al-Masadeh
ID: 20221102027
Supervisor: Dr. Sahar Qaadan
Course: Machine Intelligence I





Agenda



1 Cover

2 Agenda

3 Introduction

4 Simulations

5 Future Plans

6 Conclusions

Introduction

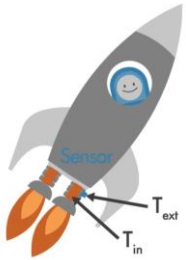
Motivation – The Need of State Estimation (The "Hidden State" Problem)

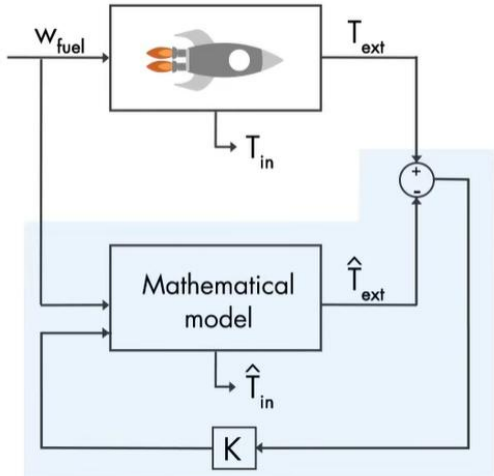
The Mission Scenario:
Goal: A spacecraft mission to the Moon (Artemis/Apollo style).
Critical System: The main engine exhaust chamber.
Requirement: We must maintain the **internal core temperature** (T_{core} (at exactly **3,000 °C**.)
If it gets too hot, the engine explodes; if it gets too cold, thrust is lost.

The Engineering Constraint:
Problem: We cannot place a sensor inside the exhaust flow.
Why? No sensor can survive **3,000 °C** and extreme pressure—it would melt instantly.
Solution: Place the sensor on the **external engine case** (T_{case}).
Another Issue:
Delayed: Heat transfer takes time to reach the casing
Noisy: Vibrations and harsh environment corrupt the signal

Main Solution:
We add a state estimator (observer)
Types of State Estimation:
- **Luenberger Observer**
- **Linear Quadratic Estimator**







STATE OBSERVER

Introduction

Literature Overview (1/2) – *Kalman, R. E. (1960). "A New Approach to Linear Filtering and Prediction Problems."*

Kalman introduced an optimal recursive estimator for linear dynamic systems with Gaussian noise, now known as the Kalman Filter.

State model:

$$x_{k+1} = F_k x_k + B_k u_k + w_k$$

Measurement Equation :

$$z_k = H_k x_k + v_k$$

Where:

x_k: State vector.

z_k: measurement.

F_k: state transition matrix.

H_k: measurement matrix.

$w_k \sim \mathcal{N}(0, Q_k)$: process noise.

$v_k \sim \mathcal{N}(0, R_k)$: measurement noise.

Prediction:

$$\hat{x}_{k|k-1} = F_k \hat{x}_{k-1|k-1} + B_k u_k$$
$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^{\top} + Q_k$$

Update:

$$K_k = P_{k|k-1} H_k^{\top} (H_k P_{k|k-1} H_k^{\top} + R_k)^{-1}$$
$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (z_k - H_k \hat{x}_{k|k-1})$$
$$P_{k|k} = (I - K_k H_k) P_{k|k-1}$$

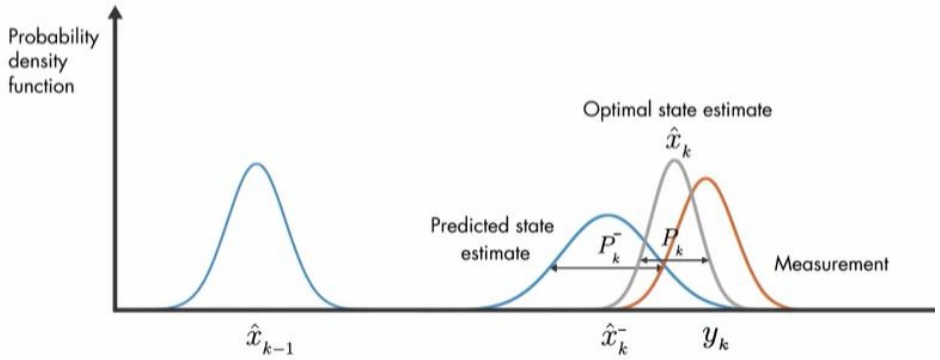


Fig.1

What it solves?

- The method provides minimum mean-square error (MMSE) state estimates by optimally combining noisy measurements with a system model, overcoming limitations of batch least-squares methods.
- The KF is optimal only for linear, Gaussian systems and requires accurate knowledge of Q and R.

Metrics :

- Mean Squared Error (MSE)
- NIS (innovation consistency)

Assumption: linear, Gaussian systems

Types of None-linear Kalman Filer:

- Extended KF, Unscented KF.

Introduction

Literature Overview (2/2) – *Akhlaghi, S., et al. (2017). "Adaptive Adjustment of Noise Covariance in Kalman Filter"*

The paper proposes an adaptive Kalman Filter framework that dynamically adjusts the process and measurement noise covariance matrices using innovation-based statistics.

State model:

$x_{k+1} = F_k x_k + B_k u_k + w_k$

Measurement Equation :

$z_k = H_k x_k + v_k$

Where:

x_k: State vector.
z_k: measurement.
F_k: state transition matrix.
H_k: measurement matrix.
$w_k \sim \mathcal{N}(0, Q_k)$: process noise.
$v_k \sim \mathcal{N}(0, R_k)$: measurement noise
v_k: Innovation
$\hat{S}_k$: actual innovation covariance.
S_k: predicted innovation covariance.

The Innovation is the **"Surprise"** factor. It is the difference between what the sensor **actually saw** and what the Kalman Filter **predicted** it would see.

Prediction:

$\hat{x}_{k|k-1} = F_k \hat{x}_{k-1|k-1} + B_k u_k$
$P_{k|k-1} = F_k P_{k-1|k-1} F_k^\top + Q_k$

Update:

$K_k = P_{k|k-1} H_k^\top (H_k P_{k|k-1} H_k^\top + R_k)^{-1}$
$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (z_k - H_k \hat{x}_{k|k-1})$
$P_{k|k} = (I - K_k H_k) P_{k|k-1}$

Adaptive Covariance Update:

Innovation: $v_k = z_k - \hat{z}_{k|k-1}$

$\hat{S}_k = v_k v_k^T$

$S_k = H P_{k|k-1} H^T + R_k$

$R_k = \beta R_{k-1} + (1-\beta)(v_k v_k^T - H_k P_{k|k-1} H_k^T)$

$Q_k = \alpha Q_{k-1} + (1-\alpha) K_k (v_k v_k^T - S_k) K_k^T$

- **Innovation :**Difference between measured and predicted output.
- **Large innovation due to sensor noise → increase** R_k
- **Large innovation due to model mismatch → increase** Q_k
- $0 < \beta < 1$controls smoothing
- α controls adaptation speed

- All KF/EKF/UKF versions assume fixed and known noise covariances.
- But real systems have:
  1. Time-varying sensor noise.
  2. Model uncertainties
  3. Sensor degradation (e.g., batteries, IMUs, cameras)
  4. Sudden nonlinear events (e.g., battery voltage knee)

Simulations

2D Tracking Test (1/2) - comparison between Adaptive Kamlan Filter & Basic Kalman Filter

two-dimensional plane under a Constant Velocity motion, where:

$x = \begin{bmatrix} p_x \\ v_x \\ p_y \\ v_y \end{bmatrix}$, p_x, p_y positions, and v_x, v_y are velocities.

$F = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix}$, State Transition Model.

$z_k = \begin{bmatrix} p_x \\ p_y \end{bmatrix} + \text{noise}$, $H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$, Measurement / Output Model

The test consists of three phases:

- **Phase 1 – Healthy Sensor (k = 0–200):**

$$R = \text{diag}(1,1)$$

- **Phase 2 – Severe Degradation (k = 200–500):**

$$R = 25I$$

- **Phase 3 – Partial Recovery (k = 500–800):**

$$R = 4I$$

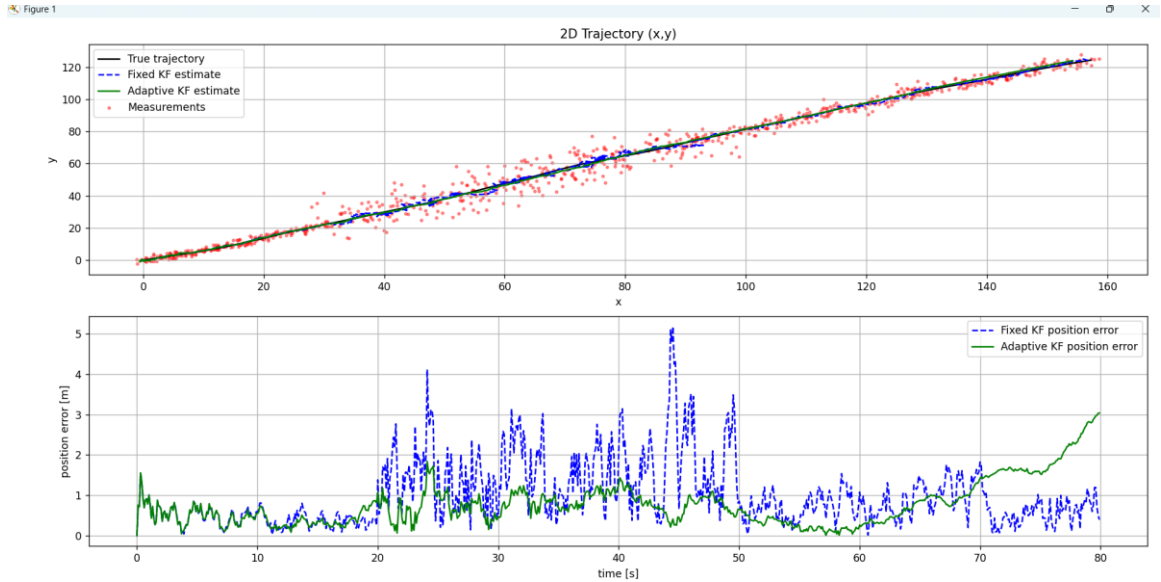


Fig.2 Trajectory Tracking

Simulations

2D Tracking (2/2) - comparison between Adaptive Kamlan Filter & Basic Kalman Filter (Results)

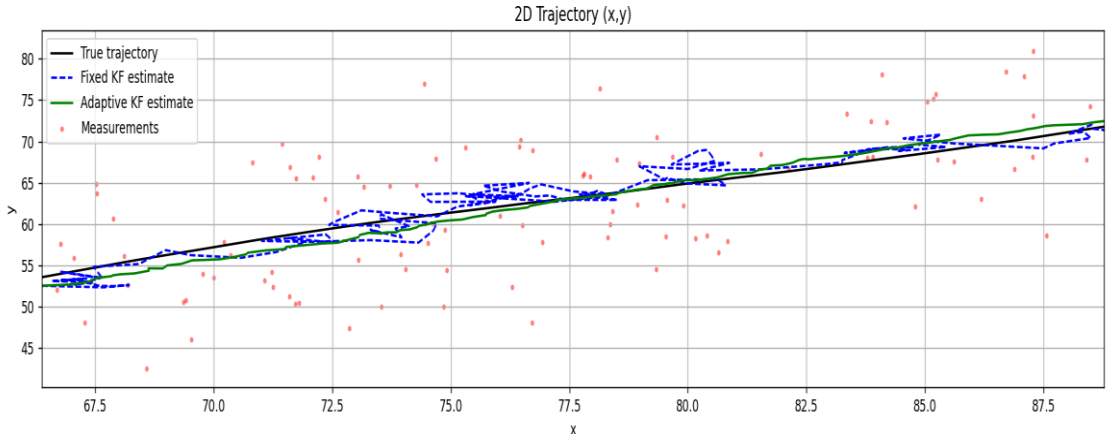


Fig.3-Trajectory Tracking (Zoomed)

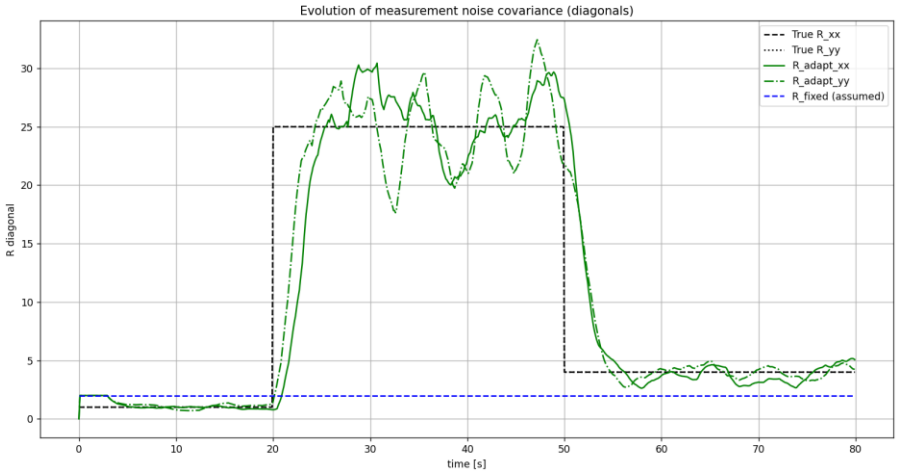


Fig.4 - Noise Estimation

Root Mean Square Error (RMSE) Comparison:

- Fixed Kalman Filter: 0.8973
- Adaptive Kalman Filter: 0.6723

Simulations

Case Study – Battery State of Charge Estimation using Dual – Unscented Adaptive Kalman Filter

- Equivalent Circuit Model (Thevenin):
- $x = \begin{bmatrix} SoC \\ V_{polarization} \end{bmatrix}$
- Control Input:
- $u = I$(Load Current)
- Measurement Function:
- $V_t = V_{oc}(SoC) - V_p - I \cdot R_0$

**where Voc (SoC)is a nonlinear 6th-order polynomial*

- State Transition Function $f(x, u)$:
- $SoC_k = SoC_{k-1} - \frac{\eta \Delta t}{C_n} I_{k-1}$

$$V_{p,k} = e^{-\Delta t/\tau} V_{p,k-1} + R_p\left(1 \rightarrow e^{-\Delta t/\tau}\right) I_{k-1}$$

Dataset: NASA Battery Data Set (B0005), featuring accelerated aging profiles at room temperature.

- The estimated voltage (red dashed line) exhibits near-perfect tracking of the true sensor voltage (black line). Crucially, the filter maintains lock during the rapid voltage collapse at $t \approx 3300s$(the "knee"), a region where standard physics-based filters typically diverge due to unmodeled electrochemical stiffness.

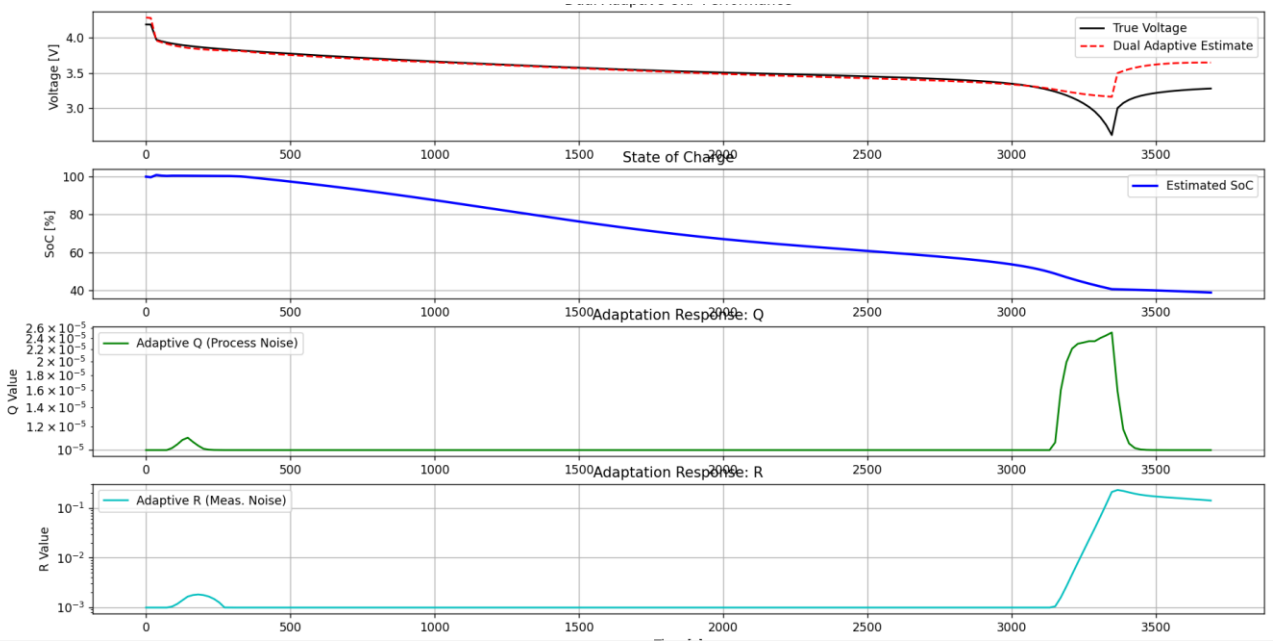


Fig.4 - DA UFK for Battery State of Charge

Simulations

Case Study – Battery State of Charge Estimation using Dual – Adaptive Kalman Filter (Results)

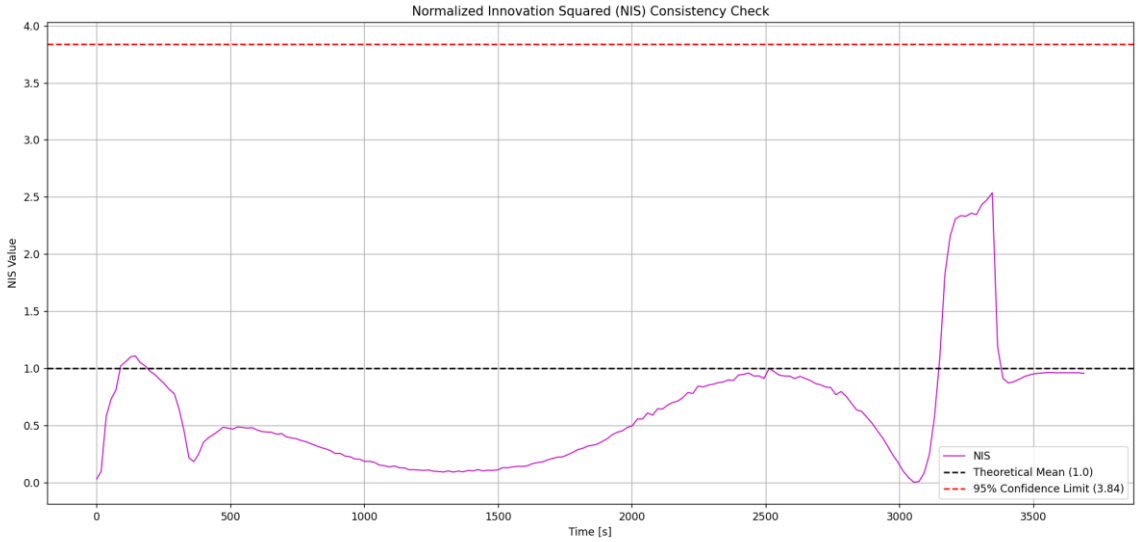
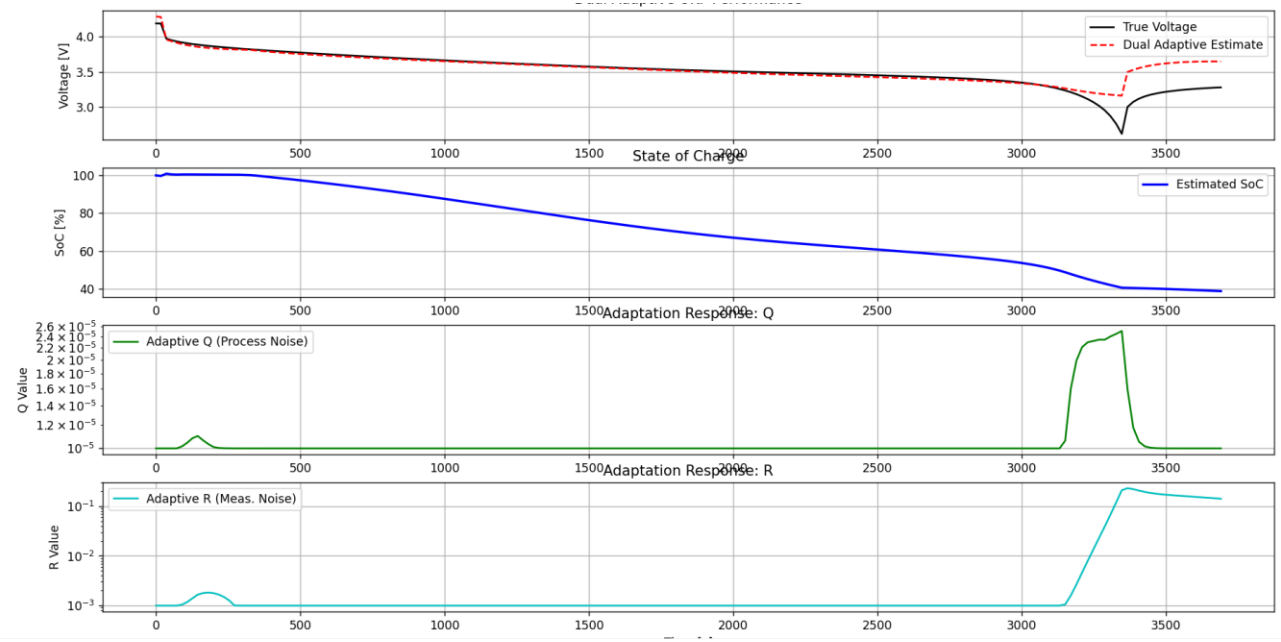


Fig.4 – Normalized Innovation Squared (NIS)



Stable Region (0–3000 s):
Throughout most of the discharge cycle, the NIS remains below its theoretical mean:

$$\mathbb{E}[\mathbb{NIS}] = n_z = 1$$

indicating a slightly conservative but stable voltage model in the linear SoC region.

Nonlinear Knee Detection ($t \approx 3300$ s):
A pronounced spike in the NIS is observed:

$$\text{NIS}_k \gg \chi^2_{0.95}$$

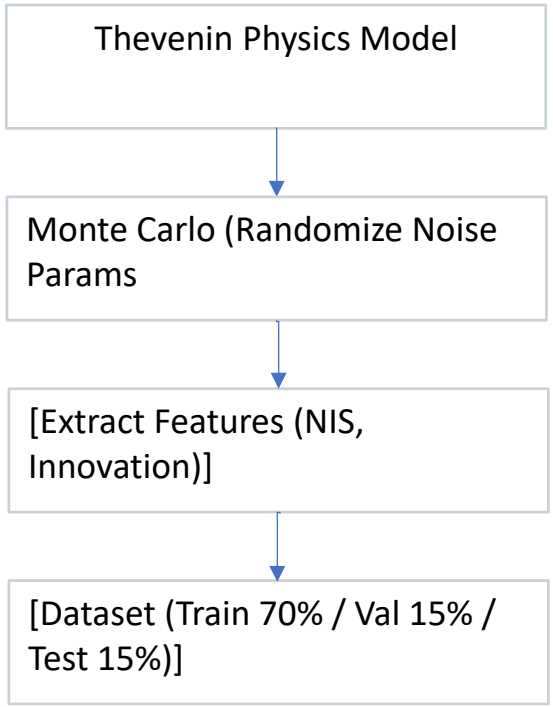
This spike coincides with the deep-discharge knee, where internal resistance increases sharply and voltage collapses nonlinearly.

Future Plans

How Simulation Creates Synthetic Data

Data generation plan (for ML Training):

1. **Exported Variables**:
   - **Inputs (Features):** Innovation (v), NIS, Voltage (V), Current (I).
   - **Targets (Labels):** True Error Magnitude, Optimal Noise Scaling Factors (Q_{scale}, R_{scale})
2. **Statistics:**
   - **#runs:** 1,000 Monte Carlo simulations
   - **Length:** 3,600 steps per run (equivalent to 1 hr discharge)
   - **Sampling rate:** 10 Hz ($\Delta t = 0.1s$)
3. **Variations:**
   - **Initial Conditions:** Randomized initial SoC ($100\% \pm 5\%$)
   - **Noise Profiles:** Injection of random "Sensor Drifts" and "Resistance Jumps" to simulate aging



Scenario	Noise Level	Parameter Change	#Runs
Baseline	Low Gaussian Noise	Ideal Parameters	200
Sensor Drift	Linearly Increasing (R)	Bias + 0.1 mV/sec	300
Model Fail	Standard (Q)	Internal Resistance	300
Combined	High (R) + High (Q)	Sudden "Knee" & Sensor Noise	200

Future Plans

Identified Research Gap

Akhlaghi, S., et al. (2017): Proposed "Innovation Covariance Matching" using a sliding window (e.g. , N = 30 samples) to estimate noise statistics.

Gap Statement (The Limitation):The "Window" Problem: Statistical methods rely on averaging past errors. This creates a **mathematical lag**. The filter only adapts *after* the failure has occurred and the window fills up.
Reactive vs. Predictive: The current DA-UKF is **reactive**—it sees a spike and fixes it. It cannot detect the subtle "precursors" of a battery voltage collapse (the "Knee") before the actual drop happens.

Proposed Contribution (AI / LSTM Integration):
Solution: Replace the statistical sliding window with an **LSTM (Long Short-Term Memory)** Neural Network.
Why LSTM fills the gap:

- **Zero-Lag Adaptation:** Detects patterns instantly without waiting for a statistical window to fill.
- **Temporal Memory:** Learns cause-and-effect over time (e.g., current load causing later voltage drop) for predictive tuning.

Future Plans

Machine Learning Plan & Features

Dataset design:
Dataset size: ~50,000 samples (Derived from 15 simulation runs × 3,600 time steps).
Split idea: Train **70%** / Validation **15%** / Test **15%**.
Data format: CSV (Time-series matrix).

Learning Task:

- **Goal:** Predicting continuous values.
- **Target / Output:** Optimal scaling factors for Process Noise (Q) and Measurement Noise (R).

Why it helps? - Instead of relying on fixed statistical formulas, the model learns battery-specific nonlinear aging patterns.

Feature	Source	Purpose
Innovation	Filter Output	The raw error signal (Model vs Sensor).
NIS Score	Filter Output	Statistical trigger for model failure.
Voltage Slope	Sensor	Indicates proximity to the "Voltage Knee."
SoC Estimate	State Vector	Context (Failures occur near 0% SoC).

Candidate Algorithm: Simple LSTM (Long Short-Term Memory).
Reason: LSTMs are lightweight, easy to implement in Python (Keras), and specifically designed to handle time-series memory (understanding that a voltage drop now relates to current usage seconds ago).

Future Plans

Real Deployment

Sensors Needed:

- **Voltage** (z_k)Voltage Divider + 12-bit ADC (Range: 0–5V).
- **Current** (u_k)Hall Effect Sensor (e.g., ACS712) or Shunt Resistor.
- **Sampling Rate**: 10 Hz ($\Delta t = 0.1s$) — matches the simulation and is standard for BMS applications.

Real Data Collection Plan:

- **Hardware Platform**: Microcontroller for handling matrix math.
- **Setup**: Controlled discharge of a generic 18650 Li-ion cell using an electronic load.
- **Storage**: Log sensor data (V, I) and estimator outputs (SoC, P_k (to an SD card in CSV format.

Expected Challenges:

- **Computational Load**: The UKF requires matrix inversion ($O(n^3)$,(which is heavy for a microcontroller; code optimization.
- **Non-Gaussian Noise**: Real electronics exhibit **60 Hz hum** and **bias drift**, violating the strict Gaussian noise assumption of the standard Kalman Filter.

Evaluation on Real System:

- **Success Metric**: The **NIS (Normalized Innovation Squared)** must remain within the **95% confidence bound** (< 3.84) during real-time operation.
- **Validation**: Periodically pause the discharge to allow the battery to rest, measure the true **Open Circuit Voltage (OCV)**, and compare it with the estimated SoC to compute **real-world RMSE**

Conclusions

Summary of Simulations

What was done:

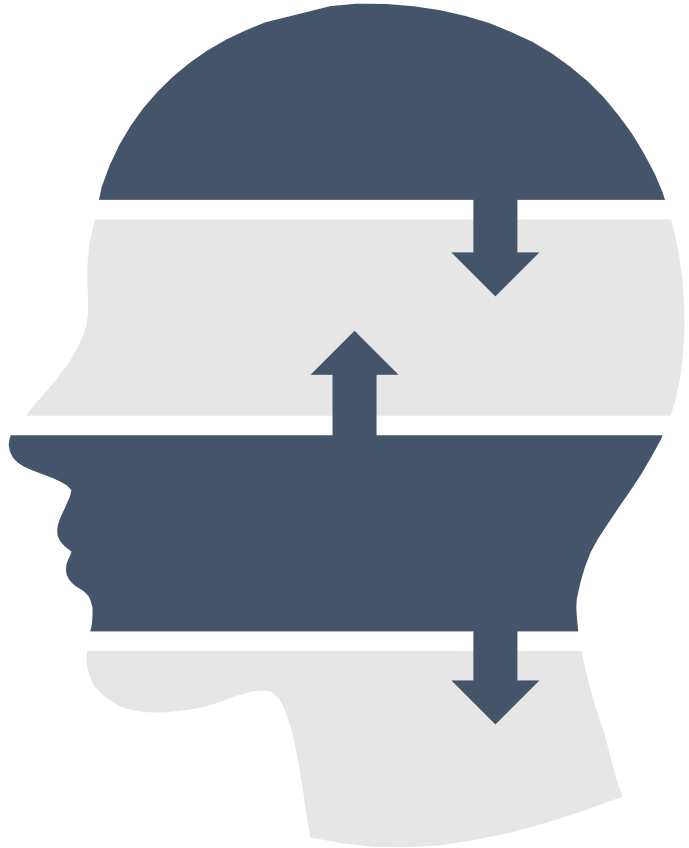
- Implemented Dual Adaptive UKF (DA-UKF) with simultaneous Q& Rtuning.
- Validated on NASA Battery Data (Non-linear) and 2D Simulation (Sensor Failure).
- Achieved ~25% RMSE reduction in tracking compared to fixed filters.

What was learned:

- **Fixed Filters Fail:** Static noise parameters cause divergence during degradation.
- **Adaptation Works:** The filter successfully switches trust between Physics and Sensors using the NIS Trigger.
- Learning the foundation of Kalman Filter, types, and applications.

What comes next:

- AI Integration: Replace statistical formulas with LSTMs for predictive (zero-lag) adaptation. Hardware: Deploy logic on STM32 microcontrollers for real-time Battery Management Systems testing.



Conclusions

References

Base paper:
Akhlaghi, S., Zhou, N., & Huang, Z. (2017). "Adaptive Adjustment of Noise Covariance in Kalman Filter for Dynamic State Estimation." *IEEE Power & Energy Society General Meeting*. DOI: 10.1109/PESGM.2017.8273756

Recent gap paper:
Chhabra, A., Venepally, J. R., & Kim, D. (2021). "Measurement Noise Covariance-Adapting Kalman Filters for Varying Sensor Noise Situations." *Sensors*, 21(24), 8304.

Other supporting papers/tools:

- **Original Theory:** Kalman, R. E. (1960). "A New Approach to Linear Filtering and Prediction Problems." *Transactions of the ASME—Journal of Basic Engineering*, 82 (Series D): 35–45.
- **Dataset:** NASA Prognostics Center of Excellence (PCoE). "Li-ion Battery Aging Datasets (B0005)." NASA Ames Research Center.
- **Battery Model:** Sun, F., et al. (2011). "State of charge estimation for lithium-ion batteries based on an adaptive unscented Kalman filter." *Energy*, 36(5)
- **Software Library:** Labbe, R. "Kalman and Bayesian Filters in Python." (filterpy).